

SecPipeLine 개발 가이드 문서 2

SecPipeline — 최종 개발 가이드

이 문서는 5개 참고 튜토리얼의 실제 내용을 기반으로 작성되었습니다. 각 단계에서 어떤 튜토리얼의 어떤 섹션을 열어서 따라해야 하는지 명시합니다.

참고 튜토리얼 목록 (즐거찾기 해주세요)

번호	튜토리얼	약칭	URL
T1	AppSignal — Flask + Docker → Render 배포	T1-AppSignal	https://blog.appsignal.com/2025/08/06/deploy-a-python-flask-app-to-render-with-docker.html
T2	(T1과 동일 링크 — 중복 제공됨)	—	—
T3	GitHub Blog — GitHub Actions 입문	T3-Actions	https://github.blog/developer-skills/github/github-for-beginners-getting-started-with-github-actions/
T4	OneUptime — Gitleaks 시크릿 스캐닝	T4-Gitleaks	https://oneuptime.com/blog/post/2026-01-25-secret-scanning-gitleaks/view
T5	Rohit Ranjan — pip-audit + CI/CD	T5-PipAudit	https://rrohitrockss.medium.com/automated-patching-of-python-dependencies-securing-your-codebase-with-pip-audit-and-ci-cd-cdd7c903e30a

0. 프로젝트 한 줄 요약

Flask 기반 학습 기록 웹앱(StudyLog)을 만들고, 5가지 의도적 보안 취약점을 삽입한 뒤, GitHub Actions 기반 5-Gate DevSecOps CI/CD 파이프라인으로 탐지(Red Run) → 수정 → 재검증(Green Run) → 배포까지 전체 흐름을 실증한다.

1. 환경 세팅

1-1. 필수 소프트웨어 설치

소프트웨어	다운로드	확인 명령
Python 3.12+	https://www.python.org/downloads/	<code>python --version</code>
Git	https://git-scm.com/downloads	<code>git --version</code>
VS Code	https://code.visualstudio.com/	<code>code --version</code>
Docker Desktop	https://www.docker.com/products/docker-desktop/	<code>docker --version</code>
GitHub CLI	https://cli.github.com/	<code>gh --version</code>

Docker 설치 참고: T1-AppSignal의 “Docker Installation” 섹션을 읽어보세요. 이 튜토리얼은 macOS M시리즈에서 Docker Desktop이 보안 인증서 문제를 일으킬 수 있다고 경고하며, 대안으로 Homebrew + Colima 설치 방법을 상세히 설명합니다.

Windows라면 Docker Desktop 공식 설치를 따르면 됩니다. macOS라면 T1의 Colima 설치 순서(`brew install docker` → `brew install colima` → `colima start` → `brew install docker-compose`)를 그대로 따라하세요.

1-2. GitHub 저장소 생성 및 클론

```
# GitHub에서 SecPipeline Private repo 생성 (README + .gitignore=Python 선택)
cd ~/Desktop
git clone https://github.com/<your-id>/SecPipeline.git
cd SecPipeline
```

```
# Python 가상환경
python -m venv venv
source venv/bin/activate      # macOS/Linux
# venv\Scripts\activate      # Windows

pip install --upgrade pip
code .
```

1-3. GitHub CLI 인증

```
gh auth login
# → GitHub.com 선택 → HTTPS → 브라우저 인증
```

2. 프로젝트 구조

아래 구조를 프로젝트 루트에 한 번에 생성합니다.

```
mkdir -p app/routes app/templates/auth app/templates/studylog app/static/css \
      tests scripts docs/notes docs/scan-results/red-run docs/scan-results/green-run \
      docs/incident-reports docs/retro .github/workflows .semgrep
touch app/__init__.py app/config.py app/models.py run.py \
      app/routes/__init__.py app/routes/auth.py app/routes/studylog.py \
      app/routes/subjects.py app/routes/search.py app/routes/dashboard.py \
      app/routes/export.py \
      tests/__init__.py tests/test_auth.py tests/test_studylog.py \
      scripts/parse_results.py \
      Dockerfile .dockerignore requirements.txt \
      .gitleaks.toml .semgrep/custom-rules.yml \
      .github/workflows/devsecops.yml \
      docs/vulnerability-plan.md
```

최종 트리:

```
SecPipeline/
├── .github/workflows/devsecops.yml
├── .semgrep/custom-rules.yml
├── .gitleaks.toml
├── app/
│   ├── __init__.py
│   ├── config.py          ← VULN-01
│   ├── models.py
│   ├── routes/
│   │   ├── auth.py
│   │   ├── studylog.py
│   │   ├── subjects.py
│   │   ├── search.py      ← VULN-03a
│   │   ├── dashboard.py   ← VULN-03b
│   │   └── export.py
│   ├── templates/...
│   └── static/...
├── tests/
├── scripts/parse_results.py
├── docs/
├── Dockerfile             ← VULN-04, VULN-05
├── .dockerignore
├── requirements.txt       ← VULN-02
├── run.py
└── README.md
```

3. 의도적 취약점 5가지

ID	취약점	위치	탐지 Gate	도구
VULN-01	하드코딩 시크릿	app/config.py	Gate 1	Gitleaks
VULN-02	취약 종속성 (CVE-2023-30861)	requirements.txt	Gate 2	pip-audit
VULN-03a	SQL Injection (f-string)	app/routes/search.py	Gate 3	Semgrep+Bandit
VULN-03b	XSS (render_template_string)	app/routes/dashboard.py	Gate 3	Semgrep+Bandit
VULN-04	오래된 베이스 이미지 + root 실행	Dockerfile	Gate 4	Trivy
VULN-05	Dockerfile 베스트프랙티스 위반	Dockerfile	Gate 5	hadolint

학술 근거 — Schuckert et al. (2023), "Insecurity Refactoring: Automated Injection of Vulnerabilities in Source Code," *Computers & Security* 128, 103121. Code Property Graph 기반으로 실제 CVE 패턴을 소스 코드에 삽입하는 방법론. 우리는 이 원리를 수동으로 적용한다.

4. 스프린트별 상세 가이드 (6주)

Sprint 1 (Week 1) — 환경 세팅 + 프로젝트 정의 (~7h)

이번 주 할 일: 위 1번(환경 세팅)과 2번(프로젝트 구조 생성)을 완료하고, docs/vulnerability-plan.md 에 위 3번 표를 마크다운으로 작성한다.

논문 읽기: Fu et al. (2024), "AI for DevSecOps," arXiv:2404.04839. §4.3-4.4만 읽고, docs/notes/sprint1-survey.md 에 다음 3가지를 메모한다: (1) DevSecOps 파이프라인 5단계 (Code Commit → Build → Test → Deploy → Monitor), (2) JIT-SDP 개념, (3) Dependency Management 영역의 AI 연구 부족.

확인 사항:

- python --version → 3.12.x
- docker --version → 정상
- git remote -v → SecPipeline repo 연결됨
- docs/vulnerability-plan.md 커밋됨
- docs/notes/sprint1-survey.md 커밋됨

커밋:

```
git add .
git commit -m "SAF-1 프로젝트 정의 + 환경 세팅 + 논문 서베이"
git push origin main
```

Sprint 2 (Week 2) — Flask 앱 구현 + 취약점 삽입 (~8h)

이번 주 읽을 튜토리얼: T1-AppSignal의 "Why Flask?", "Why Docker?" 섹션을 읽어서 Flask와 Docker를 왜 쓰는지 이해하세요. (Celery/RabbitMQ 부분은 우리 프로젝트에 불필요하니 건너뛰세요.)

논문 읽기: Schuckert et al. (2023), "Insecurity Refactoring." 취약점 삽입의 학술적 근거. docs/notes/sprint2-insecurity.md 에 핵심 3줄 요약.

Day 1-2: 앱 코드 작성

requirements.txt — 취약 버전을 의도적으로 지정 (VULN-02):

```
flask==2.2.5
jinja2==3.1.2
werkzeug==2.2.3
unicorn==21.2.0
```

```
pip install -r requirements.txt
```

app/config.py (VULN-01 — 하드코딩 시크릿):

```
import os

SECRET_KEY = "super-secret-key-12345" # VULN-01
DATABASE_PATH = os.path.join(os.path.dirname(__file__), '..', 'studylog.db')
```

app/models.py:

```
import sqlite3
from app.config import DATABASE_PATH

def get_db():
    conn = sqlite3.connect(DATABASE_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    conn = get_db()
    conn.executescript('''
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            password_hash TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS subjects (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            user_id INTEGER,
            FOREIGN KEY (user_id) REFERENCES users(id)
        );
        CREATE TABLE IF NOT EXISTS study_logs (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            content TEXT,
            hours REAL DEFAULT 0,
            subject_id INTEGER,
            user_id INTEGER,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (subject_id) REFERENCES subjects(id),
            FOREIGN KEY (user_id) REFERENCES users(id)
        );
    ''')
    conn.close()
```

app/__init__.py:

```
from flask import Flask
from app.config import SECRET_KEY
from app.models import init_db

def create_app():
```

```

app = Flask(__name__)
app.secret_key = SECRET_KEY

with app.app_context():
    init_db()

from app.routes.auth import auth_bp
from app.routes.studylog import studylog_bp
from app.routes.subjects import subjects_bp
from app.routes.search import search_bp
from app.routes.dashboard import dashboard_bp
from app.routes.export import export_bp

app.register_blueprint(auth_bp)
app.register_blueprint(studylog_bp)
app.register_blueprint(subjects_bp)
app.register_blueprint(search_bp)
app.register_blueprint(dashboard_bp)
app.register_blueprint(export_bp)

@app.route('/health')
def health():
    return {"status": "ok"}, 200

return app

```

run.py:

```

from app import create_app
app = create_app()

if __name__ == '__main__':
    app.run(debug=True, port=5000)

```

app/routes/auth.py:

```

from flask import Blueprint, render_template, request, redirect, url_for, session, flash
from werkzeug.security import generate_password_hash, check_password_hash
from app.models import get_db

auth_bp = Blueprint('auth', __name__)

@auth_bp.route('/')
def index():
    return redirect(url_for('auth.login'))

@auth_bp.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        db = get_db()
        try:
            db.execute('INSERT INTO users (username, password_hash) VALUES (?, ?)',

```

```

        (username, generate_password_hash(password)))
    db.commit()
    flash('Registration successful')
    return redirect(url_for('auth.login'))
except Exception:
    flash('Username already exists')
finally:
    db.close()
return render_template('auth/register.html')

@auth_bp.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        db = get_db()
        user = db.execute('SELECT * FROM users WHERE username = ?', (username,)).fetch
one()
        db.close()
        if user and check_password_hash(user['password_hash'], password):
            session['user_id'] = user['id']
            session['username'] = user['username']
            return redirect(url_for('studylog.list_logs'))
        flash('Invalid credentials')
    return render_template('auth/login.html')

@auth_bp.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('auth.login'))

```

app/routes/search.py (VULN-03a — SQL Injection):

```

from flask import Blueprint, request, render_template
import sqlite3
from app.config import DATABASE_PATH

search_bp = Blueprint('search', __name__)

@search_bp.route('/search')
def search():
    keyword = request.args.get('q', '')
    conn = sqlite3.connect(DATABASE_PATH)
    # VULN-03a: SQL Injection - f-string으로 직접 쿼리 조합
    results = conn.execute(
        f"SELECT * FROM study_logs WHERE title LIKE '%{keyword}%"
    ).fetchall()
    conn.close()
    return render_template('search.html', results=results, keyword=keyword)

```

app/routes/dashboard.py (VULN-03b — XSS):

```

from flask import Blueprint, render_template_string, request, session

dashboard_bp = Blueprint('dashboard', __name__)

```

```
@dashboard_bp.route('/dashboard')
def dashboard():
    username = session.get('username', 'Guest')
    # VULN-03b: XSS - 사용자 입력을 render_template_string에 직접 연결
    return render_template_string(
        "<h1>Welcome, " + username + "</h1>"
        "<p>학습 통계 대시보드</p>"
    )
```

app/routes/studylog.py, subjects.py, export.py — 표준적인 CRUD 코드. 취약점 없음. (Flask 공식 튜토리얼 <https://flask.palletsprojects.com/en/stable/tutorial/> 참고하여 작성)

app/templates/base.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>StudyLog</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
    <nav>
        <a href="{{ url_for('studylog.list_logs') }}">Home</a>
        <a href="{{ url_for('search.search') }}">Search</a>
        <a href="{{ url_for('dashboard.dashboard') }}">Dashboard</a>
        {% if session.get('user_id') %}
            <a href="{{ url_for('auth.logout') }}">Logout</a>
        {% endif %}
    </nav>
    <div class="container">
        {% with messages = get_flashed_messages() %}
            {% if messages %}
                {% for msg in messages %}<p class="flash">{{ msg }}</p>{% endfor %}
            {% endif %}
        {% endwith %}
        {% block content %}{% endblock %}
    </div>
</body>
</html>
```

app/templates/auth/login.html, register.html — base.html을 extends하는 간단한 폼 페이지.

Day 3: 로컬 실행 확인

```
python run.py
# → http://127.0.0.1:5000 접속
# → 회원가입 → 로그인 → 학습 기록 추가 → 검색 → 대시보드 확인
```

확인 사항: 모든 페이지가 200 OK로 뜨는지 확인.

Day 4: Dockerfile 작성

📖 T1-AppSignal의 “Create Your Python Flask App With Docker” 섹션을 열어두세요. 이 튜토리얼에서 Dockerfile의 각 명령어(**FROM**, **WORKDIR**, **COPY**, **RUN**, **EXPOSE**, **CMD**)가 무엇을 하는지 하나씩 설명합니다. 특히 9단계 설명(Gunicorn으로 Flask 실행)을 참고하세요.

단, 우리의 Red Run용 Dockerfile은 **의도적으로 나쁘게** 작성합니다:

```
# VULN-04: 오래된 베이스 이미지 (python:3.8-slim에는 알려진 CVE가 다수 존재)
FROM python:3.8-slim

# VULN-05: WORKDIR 미설정, 전체 복사, root 실행, HEALTHCHECK 없음
COPY . .
RUN pip install -r requirements.txt
EXPOSE 5000
CMD ["unicorn", "--bind", "0.0.0.0:5000", "run:app"]
```

T1-AppSignal과 비교하면 차이가 보입니다. T1은 `WORKDIR /app` 을 설정하고, Alpine 이미지를 쓰고, 시스템 빌드 도구도 설치합니다. 우리는 Sprint 4에서 수정할 때 T1의 패턴을 참고하여 올바른 Dockerfile을 만들 것입니다.

```
docker build -t studylog .
docker run -p 5000:5000 studylog
# → http://localhost:5000 접속 확인
```

Day 5-6: 단위 테스트

tests/test_auth.py:

```
import pytest
from app import create_app

@pytest.fixture
def client():
    app = create_app()
    app.config['TESTING'] = True
    with app.test_client() as client:
        yield client

def test_health(client):
    rv = client.get('/health')
    assert rv.status_code == 200

def test_login_page(client):
    rv = client.get('/login')
    assert rv.status_code == 200

def test_register_page(client):
    rv = client.get('/register')
    assert rv.status_code == 200
```

```
pip install pytest
python -m pytest tests/ -v
```

Day 7: 커밋

```
git add .
git commit -m "SAF-4 Flask StudyLog 앱 뼈대 + 5가지 의도적 취약점 삽입"
git push origin main
```


Sprint 3 (Week 3) — CI/CD 파이프라인 5-Gate 구축 + Red Run (~7h)

이번 주 핵심: GitHub Actions에 5개 보안 게이트를 올리고, 의도적 취약점이 전부 탐지되는 것을 확인(Red Run)한다.

논문 읽기: Li et al. (2023), "Comparison and Evaluation on SAST Tools for Java," ESEM 2023. Semgrep+Bandit 조합 선택 근거. [docs/notes/sprint3-sast.md](#) 에 핵심 3줄.

Day 1: GitHub Actions 기초 학습

 **T3-Actions** 튜토리얼을 처음부터 끝까지 따라하세요. 이 튜토리얼에서 배우는 핵심:

1. **YAML 3대 구조:** `name` (워크플로우 이름), `on` (트리거 이벤트), `jobs` (실제 작업). 이 3가지가 모든 GitHub Actions 파일의 뼈대입니다.
2. **uses vs run 의 차이:** `uses` 는 GitHub Marketplace의 미리 만들어진 액션을 가져오는 것(예: `actions/checkout@v4`), `run` 은 셸 명령어를 직접 실행하는 것(예: `pip install semgrep`). T3에서 이 차이를 명확히 설명합니다.
3. **permissions:** 액션이 repo에 접근하려면 권한이 필요합니다. T3에서 `issues: write`, `contents: read` 를 설정하는 예시를 보여줍니다.
4. **Actions 탭에서 디버깅:** T3의 "Reviewing current workflows" 섹션에서 실행된 워크플로우의 각 스텝 로그를 확인하는 방법, Re-run 방법, Disable 방법을 설명합니다. 우리 파이프라인 디버깅에 필수입니다.

T3의 "이슈 라벨링" 예제를 직접 따라해 본 후, 아래 우리 파이프라인으로 넘어가세요.

Day 2: Gate 1 — Gitleaks (Secret Scan)

 **T4-Gitleaks** 튜토리얼을 열어두세요. 우리가 가져올 내용:

로컬에서 먼저 테스트 — T4의 "Installing gitleaks" + "Basic Scanning" 섹션:

```
# macOS
brew install gitleaks
# 또는 Docker로 실행
# docker run --rm -v $(pwd):/repo zricethezav/gitleaks:latest detect --source /repo

# 로컬 스캔
gitleaks detect --source .
```

T4에 나온 출력 예시처럼, `app/config.py` 의 `SECRET_KEY = "super-secret-key-12345"` 가 탐지되어야 합니다. Finding에 파일명, 라인 번호, RuleID가 표시됩니다.

.gitleaks.toml 작성 — T4의 "Custom Configuration" 섹션을 그대로 참고:

```
[extend]
useDefault = true

[allowlist]
description = "Global allowlist"
paths = [
    ''\.env\.example'',
    ''tests/fixtures/'',
]
```

지금은 allowlist를 최소한으로 유지합니다. Sprint 5에서 false positive이 발생하면 T4의 allowlist 패턴 추가 방법(`regexes`, `commits`)을 사용해서 해결할 것입니다.

CI에 추가 — T4의 "CI/CD Integration > GitHub Actions" 섹션의 YAML을 그대로 가져옵니다:

```
secret-scan:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
```

```

with:
  fetch-depth: 0 # T4: "Full history needed for scanning all commits"
- uses: gitleaks/gitleaks-action@v2
env:
  GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

```

T4에서 `fetch-depth: 0` 을 설정하는 이유를 설명합니다: Gitleaks는 전체 Git 히스토리를 스캔하므로, shallow clone이면 과거 커밋의 시크릿을 놓칠 수 있습니다.

Day 3: Gate 2 — pip-audit (Dependency Scan)

 **T5-PipAudit 튜토리얼을 열어주세요.** 우리가 가져올 내용:

로컬에서 먼저 테스트 — T5의 "Step 1: Install and Use pip-audit":

```

pip install pip-audit
pip-audit -r requirements.txt

```

T5의 예시 출력처럼 패키지명, 버전, CVE ID, 수정 버전이 표로 나옵니다. 우리의 `flask==2.2.5` 에서 CVE-2023-30861이 탐지되어야 합니다.

JSON 리포트 생성 — T5의 "Step 5: Generate Reports":

```

pip-audit -r requirements.txt -f json -o dep-scan.json

```

CI에 추가 — T5의 "Step 4: Integrate into CI/CD > GitHub Actions Example"를 참고하되, 우리는 자동 패칭(-fix)은 하지 않고 탐지만 합니다:

```

dependency-scan:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
    with:
      python-version: "3.12"
    - run: |
        pip install pip-audit
        pip-audit -r requirements.txt -f json -o dep-scan.json || true
    - uses: actions/upload-artifact@v4
      if: always()
    with:
      name: dependency-scan
      path: dep-scan.json

```

T5에서 `peter-evans/create-pull-request@v6` 으로 자동 PR을 만드는 패턴도 나오지만, 우리 프로젝트에서는 수동으로 수정 후 기록하는 것이 평가 요소이므로 이 부분은 Sprint 4에서 참고만 합니다.

Day 4: Gate 3 — Semgrep + Bandit (SAST)

```

sast:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
    with:
      python-version: "3.12"
    - run: |

```

```

        pip install semgrep bandit
        semgrep scan --config auto --config .semgrep/ app/ --json --output semgrep.j
son || true
        bandit -r app/ -f json -o bandit.json || true
- uses: actions/upload-artifact@v4
  if: always()
  with:
    name: sast-results
    path: |
      semgrep.json
      bandit.json

```

커스텀 Semgrep 규칙 (`.semgrep/custom-rules.yml`):

```

rules:
- id: flask-render-template-string-xss
  patterns:
    - pattern: render_template_string($X + ...)
  message: "render_template_string with string concatenation - potential XSS"
  languages: [python]
  severity: WARNING
  metadata:
    cwe: ["CWE-79"]

```

학술 근거 — Bennett et al. (2024), "A Systematic Evaluation of LLMs for Generating Custom Semgrep Rules." 기본 제공 규칙 외 프로젝트별 커스텀 규칙이 탐지 범위를 넓힌다.

로컬 테스트:

```

pip install semgrep bandit
semgrep scan --config auto --config .semgrep/ app/
bandit -r app/ -f screen

```

Day 5: Gate 4 — Trivy (Container Scan) + Gate 5 — hadolint (Dockerfile Lint)

⚠️ 중요: Trivy 공급망 공격 (2026-03-19)

`aquasecurity/trivy-action` 의 77개 버전 태그 중 76개가 악성 코드로 변조되었습니다. **v0.35.0만 안전합니다.** 이 사건은 Sprint 5의 운영 장애 시나리오 #3 소재로도 활용합니다.

```

build-and-scan:
  runs-on: ubuntu-latest
  needs: [secret-scan, dependency-scan, sast]
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - run: |
        pip install -r requirements.txt pytest
        python -m pytest tests/ -v --junitxml=test-results.xml
    - run: docker build -t studylog:${{ github.sha }} .
    - name: Trivy image scan
      uses: aquasecurity/trivy-action@v0.35.0 # 유일한 안전 버전!
      with:
        image-ref: studylog:${{ github.sha }}

```

```

    format: json
    output: trivy.json
  - uses: actions/upload-artifact@v4
    if: always()
    with:
      name: build-scan-results
      path: |
        test-results.xml
        trivy.json

dockerfile-lint:
  runs-on: ubuntu-latest
  steps:
  - uses: actions/checkout@v4
  - uses: hadolint/hadolint-action@v3.3.0
    with:
      dockerfile: Dockerfile

```

Day 6: 전체 YAML 통합 + Push → Red Run

모든 Gate를 하나의 `.github/workflows/devsecops.yml` 파일에 통합합니다:

```

name: SecPipeline DevSecOps
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  secret-scan:
    # ... (위 Gate 1)
  dependency-scan:
    # ... (위 Gate 2)
  sast:
    # ... (위 Gate 3)
  build-and-scan:
    needs: [secret-scan, dependency-scan, sast]
    # ... (위 Gate 4)
  dockerfile-lint:
    # ... (위 Gate 5)

```

```

git add .
git commit -m "SAF-7 GitHub Actions 5-Gate DevSecOps 파이프라인"
git push origin main

```

📖 T3-Actions의 “Reviewing current workflows” 섹션을 참고하여 GitHub > Actions 탭에서:

- 5개 job이 모두 표시되는지 확인
- 각 job 클릭 → 스텝별 로그 확인
- Gitleaks가 `super-secret-key-12345`를 탐지했는지 확인
- pip-audit가 CVE-2023-30861을 찾았는지 확인
- Semgrep/Bandit이 SQLi/XSS를 찾았는지 확인

Day 7: Red Run 결과 저장

- 각 Gate의 스크린샷을 `docs/scan-results/red-run/` 에 저장
- JSON 아티팩트를 다운로드하여 같은 폴더에 저장
- `scripts/parse_results.py` 작성 (각 JSON을 파싱하여 요약 테이블 출력)

```
git add .
git commit -m "SAF-11 Red Run 결과 기록"
git push origin main
```

Sprint 4 (Week 4) — 취약점 수정 + Green Run (~8h)

이번 주 핵심: 각 취약점을 하나씩 수정하고, 수정할 때마다 해당 Gate가 통과하는지 확인한다. 각 수정 과정을 `docs/incident-reports/fix-XX.md` 로 기록한다.

논문 읽기: Bennett et al. (2024), Custom Semgrep Rules. `docs/notes/sprint4-semgrep.md` 에 핵심 3줄.

Fix-01: 하드코딩 시크릿 수정

참고: T4-Gitleaks의 "Remediation Workflow" 섹션. 핵심 원칙: (1) 시크릿을 즉시 교체, (2) 이전 시크릿 무효화, (3) 히스토리에서 제거(선택), (4) allowlist에 추가.

```
# app/config.py - 수정 후
import os

SECRET_KEY = os.environ.get("SECRET_KEY", "dev-only-fallback")
DATABASE_PATH = os.path.join(os.path.dirname(__file__), '..', 'studylog.db')
```

`.env.example` 생성:

```
SECRET_KEY=
```

```
git add .
git commit -m "SAF-12 Fix-01: 하드코딩 시크릿 → 환경변수"
git push origin main
# → Actions에서 Gate 1 (Gitleaks) 통과 확인
```

`docs/incident-reports/fix-01-secret.md` 작성: Before 코드, After 코드, Gate 1 통과 스크린샷, 소요 시간.

Fix-02: 취약 종속성 수정

참고: T5-PipAudit의 "Step 2: Automatically Apply Security Patches." T5에서는 `pip-audit --fix` 로 자동 패치를 보여주지만, 우리는 수동으로 버전을 올려서 수정 과정을 기록합니다.

```
# requirements.txt - 수정 후
flask>=3.1.0
jinja2>=3.1.4
werkzeug>=3.1.0
unicorn==21.2.0
```

```
pip install -r requirements.txt
pip-audit -r requirements.txt # 0 vulnerabilities 확인
git add .
git commit -m "SAF-13 Fix-02: 취약 종속성 업그레이드"
git push origin main
```

Fix-03a: SQL Injection 수정

```
# app/routes/search.py - 수정 후
@search_bp.route('/search')
def search():
    keyword = request.args.get('q', '')
    conn = sqlite3.connect(DATABASE_PATH)
    results = conn.execute(
        "SELECT * FROM study_logs WHERE title LIKE ?",
        (f"%{keyword}%",)
    ).fetchall()
    conn.close()
    return render_template('search.html', results=results, keyword=keyword)
```

Fix-03b: XSS 수정

```
# app/routes/dashboard.py - 수정 후
from flask import Blueprint, render_template, session

dashboard_bp = Blueprint('dashboard', __name__)

@dashboard_bp.route('/dashboard')
def dashboard():
    username = session.get('username', 'Guest')
    return render_template('dashboard.html', username=username)
```

app/templates/dashboard.html 생성:

```
{% extends "base.html" %}
{% block content %}
<h1>Welcome, {{ username }}</h1>
<p>학습 통계 대시보드</p>
{% endblock %}
```

Fix-04 + Fix-05: Dockerfile 수정

 **T1-AppSignal의 Dockerfile 해설을 참고.** T1에서 각 명령어의 의미를 설명하는 9단계 설명을 보면서 우리의 올바른 Dockerfile을 작성합니다:

```
# Fix-04: 최신 안전 이미지 + non-root 사용자
FROM python:3.12-slim

# Fix-05: WORKDIR 설정 (T1: "Set the working directory to /app")
WORKDIR /app

# 시스템 의존성 + curl (HEALTHCHECK용)
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*

# non-root 사용자 생성
RUN adduser --disabled-password --gecos "" appuser

# requirements.txt만 먼저 복사 (Docker 캐시 최적화 - T1과 동일 패턴)
COPY requirements.txt requirements.txt
RUN pip install --no-cache-dir -r requirements.txt
```

```
# 앱 코드 복사 (소유권 지정)
COPY --chown=appuser:appuser . .

USER appuser
EXPOSE 5000

# Fix-05: HEALTHCHECK 추가
HEALTHCHECK CMD curl -f http://localhost:5000/health || exit 1

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "run:app"]
```

`.dockerignore` 생성:

```
venv/
__pycache__/
*.pyc
.git/
docs/
tests/
*.db
.env
```

전체 Green Run

```
git add .
git commit -m "SAF-17 전체 취약점 수정 완료 - Green Run"
git push origin main
```

Actions에서 5개 Gate 모두 green 확인 → 스크린샷을 `docs/scan-results/green-run/` 에 저장.

Sprint 5 (Week 5) — 배포 + DAST + 운영 장애 시나리오 (~7h)

논문 읽기: Li et al. (2025), "SoK: Towards Effective AVR," USENIX Security '25. `docs/notes/sprint5-avr.md` 에 핵심 3줄.

Day 1-2: Render 배포

📖 T1-AppSignal의 "Deploy Your Python Flask Project to Render" 섹션 전체를 읽으세요. 단, T1은 Celery+RabbitMQ를 쓰므로 CloudAMQP/render.yaml 부분은 건너뛰고, 다음 간소화된 흐름을 따릅니다:

1. <https://dashboard.render.com> 에서 계정 생성
2. **New** → **Web Service** → **Connect GitHub** → SecPipeline 선택
3. **Runtime**: Docker (T1에서도 Docker environment를 선택합니다)
4. **Environment variables** 추가:

```
SECRET_KEY = (python -c "import secrets; print(secrets.token_hex(32))" 결과)
```

5. Deploy 클릭

6. 배포 완료 후 `https://secpipeline-xxxx.onrender.com/health` 접속 → `{"status":"ok"}` 확인

T1에서 설명하는 `render.yaml` Blueprint 방식도 선택 가능합니다. 우리 프로젝트용:

```
# render.yaml (선택 사항 - UI에서 수동 설정해도 됨)
services:
  - type: web
    name: secpipeline
    runtime: docker
```

```
plan: free
envVars:
  - key: SECRET_KEY
    sync: false
```

Day 3: CI에 Deploy + DAST 추가

Deploy job — Green Run 통과 후에만 자동 배포:

```
deploy:
  runs-on: ubuntu-latest
  needs: [build-and-scan, dockerfile-lint]
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  steps:
    - name: Deploy to Render
      run: curl -X POST "${{ secrets.RENDER_DEPLOY_HOOK }}"
```

Render Settings → Deploy Hook URL 복사 → GitHub repo Settings → Secrets → `RENDER_DEPLOY_HOOK` 추가.

DAST job — ZAP 베이스라인 스캔:

```
dast:
  runs-on: ubuntu-latest
  needs: [deploy]
  steps:
    - uses: zapproxy/action-baseline@v0.14.0
      with:
        target: "${{ secrets.RENDER_APP_URL }}"
```

`RENDER_APP_URL` = `https://secpipeline-xxxx.onrender.com` 을 GitHub Secrets에 추가.

Day 4-6: 운영 장애 시나리오 3건

장애 #1 — Gitleaks False Positive:

테스트 파일에 `test-api-key-12345` 같은 더미 값이 있으면 Gitleaks가 오탐할 수 있습니다.

📖 **T4-Gitleaks의 “Custom Configuration > allowlist” 섹션 참고.** `.gitleaks.toml` 에 추가:

```
[[rules.allowlist]]
regexes = ['test-api-key-12345']
```

T4에서는 “Baseline for Legacy Repositories” 방법도 소개합니다: `gitleaks detect --source . --report-format json --report-path .gitleaks-baseline.json` 으로 기존 탐지 결과를 베이스라인으로 설정한 뒤, 이후 스캔에서 `--baseline-path` 를 지정하면 새로운 시크릿만 탐지합니다.

장애 #2 — Docker 빌드 실패:

Python 패키지 중 C 컴파일이 필요한 것이 있으면 slim 이미지에서 `gcc` 가 없어 빌드 실패. `RUN apt-get install -y gcc` 를 추가하여 해결.

장애 #3 — Trivy 공급망 공격 (2026-03-19 실제 사건):

`aquasecurity/trivy-action` 의 76개 태그가 악성 코드로 변조된 실제 사건. `v0.35.0` 만 안전. `docs/incident-reports/incident-03-supply-chain.md` 에 사건 개요, 우리 파이프라인에 미친 영향, SHA 고정으로의 전환 과정을 기록.

각 장애를 `docs/incident-reports/incident-01.md`, `incident-02.md`, `incident-03.md` 로 기록 (발생 → 원인 분석 → 해결 → 재검증).

Sprint 6 (Week 6) — E2E 시연 + 평가 + 최종 보고서 (~8h)

Day 1-2: End-to-End 시연

취약 브랜치([vuln-demo](#))에서 Red Run → main에서 Green Run → 자동 배포 → ZAP 스캔까지 전체 흐름을 한 번에 돌려 스크린샷/녹화.

Day 3-4: 정량 평가표

Gate	도구	Red Run 탐지 수	심각도	Green Run 탐지 수	수정 소요 시간	CWE
1	Gitleaks	1	HIGH	0	?분	CWE-798
2	pip-audit	?	HIGH	0	?분	CVE-2023-30861
3	Semgrep+Bandit	?	HIGH	0	?분	CWE-89, CWE-79
4	Trivy	?	CRITICAL	0	?분	—
5	hadolint	?	WARNING	0	?분	DL3xxx

Day 5-6: 최종 보고서 (7장 구조)

1. **Introduction** — 프로젝트 동기, 목표, 범위
2. **Related Work** — 5편 논문 요약 (Fu 2024, Schuckert 2023, Li 2023, Bennett 2024, Li 2025)
3. **System Design** — 아키텍처 다이어그램, 5-Gate 파이프라인 설계
4. **Implementation** — 기술 스택, 주요 코드, CI/CD YAML
5. **Incident Reports** — 취약점 수정 5건 + 운영 장애 3건 요약
6. **Evaluation** — 정량 평가표, Red/Green Run 비교
7. **Conclusion** — 배운 점, 한계, 향후 개선

Day 7: 제출

```
git tag v1.0
git push --tags
```

JIRA 스프린트 종료, [README.md](#) 정리.

5. 논문 목록 정리

Sprint	논문	프로젝트 적용
1	Fu et al. (2024), "AI for DevSecOps," arXiv:2404.04839	파이프라인 설계 근거
2	Schuckert et al. (2023), "Insecurity Refactoring," C&S 128	취약점 삽입 방법론
3	Li et al. (2023), "SAST Comparison," ESEM 2023	Semgrep+Bandit 조합 근거
4	Bennett et al. (2024), "Custom Semgrep Rules"	커스텀 규칙 작성 근거
5	Li et al. (2025), "SoK: AVR," USENIX Security '25	수정 방법 분류 + 평가

6. 튜토리얼 → 스프린트 매핑 요약

튜토리얼	읽는 시점	참고 섹션	적용 대상
T1-AppSignal	Sprint 2 Day 4 (Dockerfile 작성)	"Docker Installation", Dockerfile 9단계 해설	Dockerfile 구조, 각 명령어 의미
T1-AppSignal	Sprint 4 Day (Dockerfile 수정)	Dockerfile 해설 + WORKDIR , COPY 패턴	올바른 Dockerfile 작성
T1-AppSignal	Sprint 5 Day 1-2 (Render 배포)	"Deploy Your Python Flask Project to Render", render.yaml Blueprint	Render 배포 흐름

튜토리얼	읽는 시점	참고 섹션	적용 대상
T3-Actions	Sprint 3 Day 1 (CI 구축 전)	전체 (name/on/jobs, uses vs run, permissions, Actions 탭 디버깅)	GitHub Actions YAML 작성 기초
T4-Gitleaks	Sprint 3 Day 2 (Gate 1 구현)	Installing, Basic Scanning, Custom Configuration, CI/CD > GitHub Actions	Gitleaks 설치·스캔·CI 통합 ·gitleaks.toml
T4-Gitleaks	Sprint 4 (Fix-01)	Remediation Workflow	시크릿 수정 절차
T4-Gitleaks	Sprint 5 (장애 #1)	Custom Configuration > allowlist, Baseline	False positive 해결
T5-PipAudit	Sprint 3 Day 3 (Gate 2 구현)	Step 1 (Install), Step 4 (GitHub Actions), Step 5 (Reports)	pip-audit 설치·CI 통합·JSON 리포트
T5-PipAudit	Sprint 4 (Fix-02)	Step 2 (-fix 참고)	종속성 업그레이드 근거

7. 핵심 명령어 치트시트

```
# — 로컬 개발 —
python run.py                                # Flask 실행
python -m pytest tests/ -v                    # 테스트

# — Git —
git add . && git commit -m "SAF-XX 설명" && git push origin main

# — Docker —
docker build -t studylog .
docker run -p 5000:5000 studylog

# — 로컬 보안 스캔 —
gitleaks detect --source .                    # Gate 1
pip-audit -r requirements.txt                 # Gate 2
semgrep scan --config auto app/               # Gate 3
bandit -r app/ -f screen                      # Gate 3

# — 환경변수 생성 —
python -c "import secrets; print(secrets.token_hex(32))"
```

이 가이드가 최종본입니다. Sprint 2의 Flask 코드 작성부터 시작하시면 되고, 각 단계에서 해당 튜토리얼의 지정된 섹션을 직접 열어 따라하면서 진행하세요. 막히는 부분이 생기면 에러 메시지와 함께 알려주세요.